

# Motor de Pré-processamento Baseado no Padrão *Facade*: Uma Abordagem Nativa em Python para o Dataset Spotify

Carlos Henrique S. S. Santiago<sup>1</sup>, Gustavo B. Nonato<sup>2</sup>, Hudnei S. P. Santana<sup>3</sup>, João Guilherme G. Pinheiro<sup>4</sup>

<sup>1</sup>Curso de Sistemas de Informação – Centro Universitário de Excelência (UNEX) Feira de Santana – BA – Brasil

{241031357,241030283, 242030118, 241031134}@aluno.unex.edu.br

**Resumo.** Este artigo descreve o desenvolvimento de uma biblioteca de pré-processamento de dados em Python nativo, desenvolvida para a Dende Softhouse. O objetivo é higienizar um dataset do Spotify com 8.582 registros para alimentar um futuro sistema de recomendação. A arquitetura utiliza o padrão de projeto *Facade* para orquestrar operações de remoção de duplicatas, imputação de dados ausentes, transformações de escala (*Min-Max* e *Z-Score*) e codificações categóricas (*One-Hot* e *Label Encoding*), utilizando estritamente dicionários e listas.

**Abstract.** This article describes the development of a data preprocessing library in native Python, developed for Dende Softhouse. The objective is to sanitize a Spotify dataset with 8,582 records to feed a future recommendation system. The architecture uses the *Facade* design pattern to orchestrate operations of duplicate removal, missing data imputation, scale transformations (*Min-Max* and *Z-Score*), and categorical encodings (*One-Hot* and *Label Encoding*), strictly using dictionaries and lists.

## 1. Introdução

### 1.1.Contextualização e Justificativa:

Sistemas de recomendação são componentes críticos em plataformas digitais modernas, tendo como principal função auxiliar os usuários na complexa tarefa de tomada de decisão e descoberta de conteúdo. Para que um motor de recomendação seja eficiente — como o projetado para a Dende Softhouse —, a precisão do aprendizado de máquina depende estritamente da qualidade dos dados que o alimentam. O pré-processamento constitui a espinha dorsal desse fluxo, evitando que bases brutas e ruidosas propaguem vieses e frustrem o usuário final com sugestões irrelevantes.

Neste contexto, este artigo descreve o desenvolvimento de uma biblioteca de pré-processamento de dados em Python nativo para higienizar um *dataset* do Spotify com 8.582 registros. A arquitetura utiliza o padrão de projeto *Facade* para orquestrar operações de remoção de duplicatas, imputação de dados ausentes, transformações de escala (*Min-Max* e *Z-Score*) e codificações categóricas (*One-Hot* e *Label Encoding*). Operando estritamente com dicionários e listas, a solução converte dados inconsistentes em estruturas limpas e compreensíveis, prontas para alimentar o futuro sistema de recomendação.

### 1.2 Definição do Problema:

Bases de dados do mundo real apresentam uma série de anomalias que inviabilizam o uso direto em algoritmos de Machine Learning. Durante a análise exploratória, é comum identificar problemas críticos, tais como:

- **Registros Duplicados:** Entradas repetidas que podem viciar o modelo, atribuindo peso estatístico indevido a determinados comportamentos ou preferências.
- **Dados Ausentes (NaN):** Lacunas de informação que, se simplesmente descartadas, podem resultar em perda de contexto valioso. É necessária a aplicação de técnicas de imputação (como média ou mediana) para preservar a integridade do conjunto.
- **Discrepância de Escalas:** Variáveis numéricas frequentemente operam em ordens de grandeza distintas. Sem a devida normalização ou padronização, algoritmos matemáticos tendem a ser enviesados, atribuindo maior importância a variáveis com valores absolutos maiores.
- **Variáveis Categóricas:** Dados textuais (como gêneros musicais ou categorias de eventos) não são processados nativamente por algoritmos matemáticos, exigindo conversão para representações numéricas através de técnicas de codificação (*Encoders*).

### 1.3 Objetivos e Escopo Técnico:

O objetivo é desenvolver uma biblioteca customizada para higienizar um *dataset* do Spotify com 8.582 registros para a Dendê Softhouse. O grande diferencial técnico é a restrição de implementação: a solução foi construída utilizando estritamente recursos nativos da linguagem Python (dicionários e listas), sendo expressamente vedado o uso de bibliotecas como *Pandas*, *NumPy* ou *Scikit-Learn*.

## 2. Fundamentação Teórica

Esta seção estabelece as bases conceituais que norteiam as decisões arquiteturais e algorítmicas adotadas no desenvolvimento da biblioteca de pré-processamento. A fim de garantir a robustez e a confiabilidade das transformações aplicadas ao *dataset*, as técnicas aqui descritas fundamentam-se na literatura acadêmica e nas melhores práticas da engenharia de software e ciência de dados.

### 2.1.O Padrão de Projeto Facade

Segundo Gamma *et al.* (1994), o padrão *Facade* fornece uma interface unificada para um subsistema complexo. Sua aplicação na classe principal **Preprocessing** permite ocultar do usuário final toda a complexidade estrutural de manipulação de listas e dicionários nativos, oferecendo chamadas de métodos limpas e diretas.

### 2.2.Tratamento de Valores Ausentes

A presença de dados ausentes (*missing values*) é um fenômeno intrínseco a bases de dados reais e, se negligenciado, compromete a validade dos modelos preditivos (Little; Rubin, 2019). Existem duas abordagens fundamentais para lidar com esse problema:

- **Exclusão (Deleção *Listwise* / *Dropna*):** Consiste no descarte sumário de toda a linha (registro) que contenha ao menos um valor nulo. Embora mantenha a fidelidade absoluta dos dados remanescentes, pode resultar em uma perda massiva de informações cruciais para o algoritmo, reduzindo drasticamente o poder estatístico do conjunto, especialmente se os dados não faltarem de forma completamente aleatória.
- **Imputação (*Fillna*):** Refere-se à substituição do valor ausente por uma estimativa estatística extraída da própria distribuição da variável, como a média, a mediana ou a

moda. A imputação preserva o volume total do *dataset*. Em distribuições assimétricas ou com presença de valores atípicos (*outliers*), a imputação pela mediana é teoricamente preferível à média, por ser uma medida de tendência central mais robusta.

### 2.3.Transformadores de Escala

Algoritmos de aprendizado de máquina, especialmente aqueles baseados em cálculo de distâncias ou otimização por gradiente descendente, são criticamente sensíveis à magnitude das grandezas numéricas (Han; Kamber; Pei, 2011). Em um *dataset* como o do Spotify, uma variável como "quantidade de reproduções" dominaria matematicamente o peso do modelo em detrimento de uma variável como "duração em minutos" (valores baixos), criando um viés indesejado. Para nivelar essas grandezas, aplicam-se transformadores matemáticos:

- **Normalização (Min-Max Scaler):** Redimensiona os dados linearmente para que caibam em um intervalo fixo e predefinido, geralmente entre 0 e 1. A transformação é dada pela fórmula:

$$X_{scaled} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Onde:

- $X_{scaled}$ : É o novo valor do dado após a normalização (o resultado final que ficará entre 0 e 1).
  - $X$ : É o valor original do registro atual (por exemplo, a duração exata de uma música específica no dataset).
  - $X_{min}$ : É o menor valor existente em toda a coluna analisada.
  - $X_{max}$ : É o maior valor existente em toda a coluna analisada.
- **Padronização (Z-Score):** Transforma os dados de modo que a distribuição resultante possua média zero ( $\mu = 0$ ) e desvio padrão igual a um ( $\sigma = 1$ ). Esta técnica é matematicamente mais resistente à presença de *outliers* do que a normalização clássica. A fórmula aplicada a cada valor da variável é:

O diagrama mostra a fórmula  $Z = \frac{x - \mu}{\sigma}$  com setas azuis apontando para cada termo: 'Observed value' aponta para  $x$ , 'Mean value' aponta para  $\mu$ , 'z-Score' aponta para  $Z$ , e 'Standard deviation' aponta para  $\sigma$ .

$$Z = \frac{x - \mu}{\sigma}$$

Onde:

- $Z$ : É o valor padronizado, que indica a quantos desvios padrões o dado original está distante da média da coluna.
- $X$ : É o valor original do registro atual.

- $\mu$  (Mi): Representa a média aritmética de todos os valores presentes naquela coluna.
- $\sigma$  (Sigma): Representa o desvio padrão de todos os valores daquela coluna (que mede o grau de dispersão dos dados em relação à média).

## 2.4.Codificadores Categóricos (*Encoders*)

Kuhn e Johnson (2013) enfatizam a conversão de textos em representações numéricas. O *Label Encoding* atribui inteiros sequenciais (ex: 0, 1, 2), porém pode gerar falsa hierarquia matemática. Já o *One-Hot Encoding* isola as categorias em novas colunas binárias (0 ou 1), garantindo total neutralidade.

## 3. Metodologia e Arquitetura da Solução

### 3.1.A Estrutura de Dados Subjacente

Na ausência de objetos otimizados como o *DataFrame* da biblioteca *Pandas*, foi necessário arquitetar uma estrutura de dados customizada capaz de simular o comportamento de uma tabela bidimensional e suportar os 8.582 registros do *dataset* do Spotify.

A abordagem adotada consistiu na conversão do arquivo CSV original para um **dicionário Python (*Hash Map*) focado em colunas**. Nesta estrutura:

- As **chaves (*keys*)** do dicionário são *strings* que representam os nomes das colunas originais (ex: 'track\_name', 'popularity', 'duration\_ms').
- Os **valores (*values*)** associados a cada chave são **listas (*lists*)** contendo os dados da respectiva coluna.

A integridade do conjunto de dados é garantida por uma regra estrita: todas as listas armazenadas no dicionário possuem exatamente o mesmo tamanho (comprimento). Dessa forma, o índice *i* de qualquer lista corresponde à *i*-ésima linha (registro) do *dataset*, permitindo o acesso cruzado entre colunas durante as iterações e transformações matemáticas.

### 3.2.Arquitetura do Sistema: O Motor de Processamento

Para evitar a criação de um código procedural monolítico e de difícil manutenção, o sistema foi dividido em classes com responsabilidades únicas e bem definidas. O ecossistema de processamento é composto por uma classe orquestradora principal e três classes especialistas.

O sistema foi modularizado utilizando Programação Orientada a Objetos:

- **A Classe Orquestradora Preprocessing:** Materializa o *Facade*, recebendo o dicionário inicial e delegando tarefas via métodos de alto nível às classes especialistas.
- **A Classe MissingValueProcessor:** Responsável por varrer iterativamente as colunas para executar exclusão ou imputação calculando a média/mediana do zero.
- **A Classe Scaler:** Implementa as fórmulas *Min-Max* e *Z-Score* através de laços de repetição que calculam o valor mínimo, máximo, soma e variância nativamente.

- **A Classe Encoder:** Aplica o *Label Encoding* reescrevendo listas originais com inteiros e o *One-Hot* injetando novas chaves binárias no dicionário.

## 4. Análise Exploratória e Resultados

### 4.1. Perfilamento dos Dados Brutos (EDA)

Antes de aplicar qualquer transformação matemática, é mandatório compreender o estado original da base de dados. Como o uso de bibliotecas analíticas prontas foi vetado, desenvolveu-se uma classe auxiliar *Statistics*, encarregada de varrer os dicionários e calcular métricas descritivas através de iterações nativas.

O perfilamento inicial revelou uma base composta por **8.582 registros (linhas)** distribuídos em **15 colunas originais**. A análise de integridade detectou uma anomalia severa: a coluna `artist_genres` possuía **3.361 valores ausentes (Nulos/Vazios)**, o que representava quase 40% do total de dados daquela variável.

Além disso, a análise descritiva escancarou a discrepância de escalas entre as variáveis numéricas. Por exemplo:

- A variável `popularity` operava em uma escala de 0 a 100, com média em torno de 45 e desvio padrão moderado.
- Em contrapartida, a variável `duration_ms` (duração em milissegundos) apresentava valores na casa das centenas de milhares (ex: média de 230.000 ms e desvios na casa dos 50.000 ms). Essa diferença colossal de ordem de grandeza fatalmente enviesaria qualquer modelo de *Machine Learning* não tratado.

### 4.2. Execução do Pipeline

Sob orquestração da classe *Preprocessing*, o fluxo de higienização cumpriu três etapas rigorosas:

- **Tratamento de Nulos:** Evitou-se o descarte de 3.361 linhas ao imputar os valores vazios da lista `artist_genres` com a *string* "Desconhecido", preservando o volume de dados.
- **Transformação de Escala:** A classe *Scaler* aplicou padronização (*Z-Score*) em variáveis de alta magnitude (ex: `duration_ms`) e normalização (*Min-Max*) em índices limitados (`popularity`), equalizando os pesos matemáticos para o algoritmo.
- **Codificação Categórica:** Utilizou-se o *One-Hot Encoding* para converter variáveis nominais. O algoritmo criou novas chaves binárias (0 e 1) no dicionário para representar as categorias, excluindo a coluna textual original.

### 4.3. O Resultado Final

A execução do *pipeline* ocorreu com sucesso, encapsulando toda a complexidade algorítmica em um tempo de processamento otimizado, mesmo utilizando estruturas primitivas do Python. Abaixo, apresenta-se o quadro comparativo consolidado gerado pela saída do sistema:

**Quadro 1: Resumo da Transformação do *Dataset***

| Métrica                | Pré-Processamento (Dados Brutos)       | Pós-Processamento (Dados Higienizados)             |
|------------------------|----------------------------------------|----------------------------------------------------|
| Total de Registros     | 8.582                                  | 8.582 (Preservação total do volume)                |
| Total de Colunas       | 15                                     | 17 (Expansão gerada pelo <i>One-Hot Encoding</i> ) |
| Valores Ausentes (NaN) | 3.361 (em <code>artist_genres</code> ) | 0 (Imputados com "Desconhecido")                   |
| Escala Numérica        | Discrepante (0 a > 300.000)            | Padronizada/Normalizada (Média ~0, Desvio ~1)      |
| Formato Categórico     | Textos puros ( <i>Strings</i> )        | Matrizes Numéricas Binárias ( <i>Encoders</i> )    |

O *dataset* final, agora expandido para 17 colunas estritamente numéricas e padronizadas, sem nenhuma lacuna de informação, encontra-se em estado da arte. As 8.582 linhas estão perfeitamente legíveis, convertidas de sua forma bruta para tensores de dados prontos para alimentar e treinar o motor de recomendação da Dende Softhouse com máxima eficiência e sem vieses de escala.

## 5. Considerações Finais

O desenvolvimento desta biblioteca demonstrou a viabilidade de estruturar rotinas complexas de higienização de dados utilizando exclusivamente recursos nativos do Python. O padrão de projeto *Facade* foi crucial para encapsular a complexidade algorítmica da manipulação de dicionários e listas, oferecendo uma interface limpa e manutenível.

A aplicação no *dataset* do Spotify obteve êxito absoluto. A base original, antes com milhares de dados ausentes e discrepâncias de escala, foi higienizada sem a perda de nenhum dos 8.582 registros. As variáveis numéricas foram normalizadas e padronizadas, e os dados textuais convertidos em matrizes binárias através de codificadores categóricos.

Conclui-se que o *dataset* atende aos rigorosos critérios de qualidade exigidos por algoritmos matemáticos. O conjunto encontra-se estruturado, livre de vieses e inconsistências,

consolidando uma base robusta para treinar o motor de recomendação da Dende Softhouse e garantir assertividade nas sugestões aos usuários finais.

## 6. Referências Bibliográficas

GAMMA, E. *et al.* **Design Patterns: Elements of Reusable Object-Oriented Software.** Addison-Wesley, 1994.

HAN, J.; KAMBER, M.; PEI, J. **Data Mining: Concepts and Techniques.** 3. ed. Morgan Kaufmann, 2011.

KUHN, M.; JOHNSON, K. **Applied Predictive Modeling.** Springer, 2013.

LITTLE, R. J. A.; RUBIN, D. B. **Statistical Analysis with Missing Data.** 3. ed. Wiley, 2019.

RANGI, Kamlesh Kumar. **How Min-Max Scaler Works: The math behind the min-max scaler.** Medium, 25 ago. 2024. Disponível em: <https://medium.com/@iamkamleshrangi/how-min-max-scaler-works-9fbabb9347da>. Acesso em: 23 mar. 2026.

VOLK-JESUSSEK, Hannah. **z-Score: Definition, Formula, Calculation & Interpretation.** Numiqo, 27 jan. 2026. Disponível em: <https://numiqo.com/tutorial/z-score>. Acesso em: 23 mar. 2026.